

S.NO	CONTENTS	PGNO
i	ABSTRACT	6
ii	TABLE OF CONTENTS	7
iv	LIST OF ABBREVIATIONS	8
1	INRODCTION 1.1 Introduction 1.2 Problem statement 1.3 Scope of research 1.4 Existing System 1.5 Proposed System	9-13 9 10 10-11 11-12 12-13
2	LITERATURE REVIEW	14-15
3	METHODOLOGIES 3.1 System Architecture 3.2 Algorithms	16-21 16-19 20-21
4	SOFTWARE REQUIREMENTS SPECIFICATIONS 4.1 Software requirements 4.2 Hardware requirements 4.3 Languages / Technologies used	22-26 22-23 23-25 25-26
5	IMPLEMENTATIONS AND RESULT 5.1 Code 5.2 Output	27-55 56
6	RESULT	57-59
7	CONCLUSION	60
8	FUTURE ENHANCEMENTS	61
9	REFERENCES	62

ABSTRACT

The term deep-fake emerged towards the end of 2017 within the reddit community a popular American social media platform focused on content rating and discussions since then deep-fake technology has rapidly advanced enabling the crafting incredibly lifelike media material that convincingly mimics individuals appearances however its potential for misuse including the spread of disinformation and deception of users has sparked major apprehensions in response researchers have been actively exploring novel approaches to develop effective deep-fake detection systems this summary provides an overview of the latest advancements in employing machine learning techniques for deep-fake detection aiming to mitigate the risks associated with its misuse.

The proposed approach trains machine learning models on a broad dataset that contains both real and deep-fake material it uses essential characteristics taken from eye movements and facial expressions to distinguish between real and fake information the focus of the study is on using deep neural networks and making sure that they can adapt to new developments in deep-fake generating technology.

The outputs of this research seek to contribute to the development of practical and ethical deep fake detection technologies. As the threat landscape advances, it is vital to be at the forefront of technical innovations to prevent the malicious use of synthetic media. This research gives useful information for the continuing attempts to limit the hazards related with deep fake propagation.

Keywords: Deepfake Detection, Multi-task Cascaded Convolutional Networks (MTCNN), inceptionresnetv1, grad cam, Gardio.

LIST OF FIGURES

FIG NO.	FIG NAME	PG NO.
3.1.2	Deep fake generational model	17
3.1.3(a)	Process of MTCNN	17
3.1.3 (b)	Code for confidence score	19
6.1	Example input images	57
6.2 (a)	Predictable label indication for 6.1	58
6.2 (b)	Input face with an overlaid heat map by Grad Cam	58
6.3	An AI generated image	58
6.4	Predictable label indication	59
6.4 (a)	Input face with overlaid heat map by Grad CAM	59

LIST OF ABBREVIATIONS

- MTCNN-Multi-task Cascading Neutral network
- GAN- Generative Adversarial Network
- CNN- Convolutional neutral network
- MS-network -Management system
- Grad CAM- Gradient-weighted class activation mapping
- LBP-Local binary patterns
- LSTM-Long short term memory
- MFCC- Mel-frequency cepstral coefficients
- IDE- Integrated development environment
- CPU- Central processing unit
- GPU-Graphics processing unit
- CUDA- Compute unified device Architecture
- SSD-solid state drives
- HDD-Hard disk drives
- PGGA field-Programmable gate array
- ASIC-Application specific integrated circuits
- GUI- Graphic user interface

CHAPTER 1

INTRODUCTION

1.1 About Deep Fake Detection

In recent years the advancement of deep learning technology has permitted the fabrication of hyper-realistic audio visual material known as deep fakes these sophisticated manipulations powered by artificial intelligence algorithms have sparked considerable concerns owing to their potential for destructive purposes like the dissemination of misinformation and the weakening of confidence in digital media as the borders between truth and fabrication blur the requirement to build effective deep fake detection algorithms becomes increasingly clear this research article investigates the environment of deep fake detection using machine learning seeking to contribute to the creation of effective and adaptable solutions in the face of developing synthetic media threats deep fakes driven by generative adversarial networks GANS and deep neural networks have been increasingly available permitting anybody with modest technological competence to make convincing forgeries of audio video and even literary content with repercussions for several fields like politics entertainment and cyber security the necessity to prevent the harmful use of deep fakes has gotten crucial the beginning of this study article describes the importance of the deep fake phenomena stressing its prospective societal effect secondly it underscores the requirement to strengthen detection systems to ensure the integrity of digital information as we traverse a future dominated by AI-generated content recognizing and minimizing the dangers associated with deep fakes is vital to sustaining confidence in the digital domain this project seeks to add to this critical conversation by exploring and proposing unique ways for the reliable identification of deep fakes via the perspective of machine learning.

This study aims to address several key objectives:

- 1) Understanding the Deep-fake Phenomenon:** By examining the evolution and proliferation of deep-fake technology, this research seeks to gain insight into the underlying mechanisms driving its growth and impact on society.
- 2) Strengthening Detection Systems:** Through the exploration of machine learning approaches, this study aims to enhance the effectiveness of deep-fake detection systems by leveraging diverse datasets and key elements extracted from facial expressions, eye movements, and speech patterns.
- 3) Ethical Considerations:** Ethical questions related to privacy, consent, and the responsible use of deep-fake detection technologies are integral to this research. By addressing these concerns, we strive to ensure that our methodologies adhere to ethical principles and safeguard individual rights.

4) Continual Improvement: Recognizing the dynamic nature of the deep-fake landscape, this research emphasizes the importance of continual improvement in detection algorithms. By staying ahead of evolving deep-fake tactics, we aim to develop detection systems that are resilient to emerging threats.

1.2 Problem Statement

The problem of deep fake detection involves identifying manipulated media, typically videos or images, where artificial intelligence techniques, particularly deep learning, are used to alter content to depict events or scenarios that did not occur in reality. Detecting deep fakes is challenging because these AI-generated media can be highly realistic, making it difficult for humans to distinguish between real and fake content.

Key challenges in deep fake detection include:

Realism: Deep fakes are designed to be visually and audibly convincing, mimicking the natural variability and imperfections of real media.

Variability: There are numerous techniques for creating deep fakes, ranging from face swapping to voice synthesis, each requiring tailored detection methods.

Adversarial Evolution: As detection methods improve, so do the techniques used to create deep fakes, leading to a constant arms race between creators and detectors.

Ethical Concerns: Misuse of deep fakes can lead to misinformation, defamation, or manipulation of public opinion, highlighting the importance of reliable detection methods.

1.3 Scope of Research

The scope of research in deep fake detection is vast and continually evolving due to the rapid advancements in both deep fake creation techniques and detection methodologies. Here are some key areas within the scope of research in deep fake detection:

Image and Video Analysis: Research focuses on developing algorithms to detect anomalies in images and videos that indicate manipulation. This includes analysing facial features, eye movements, blinking patterns, and inconsistencies in lighting and shadows.

Audio Analysis: Detection methods also extend to audio, where researchers aim to identify unnatural fluctuations in voice pitch, cadence, and spectral characteristics that may indicate synthesized or manipulated speech.

Behavioural Biometrics: Leveraging behavioural biometrics such as typing patterns, mouse movements, or gait analysis to detect inconsistencies that suggest a deep fake in non-visual contexts.

- **Data Forensics:** Developing forensic techniques to detect traces left by deep fake generation processes, such as artificial facts in image or video metadata, compression inconsistencies, or traces of editing software.
- **Machine Learning and AI:** Utilizing machine learning models, particularly deep neural networks, for both generative adversarial networks (GANs) used in creating deep fakes and classifiers used in detecting them. This includes exploring novel architectures, feature extraction methods, and training strategies.
- **Dataset Creation and Benchmarking:** Curating diverse datasets of authentic and manipulated media to train and evaluate detection algorithms. Establishing benchmarks for performance comparison among different detection approaches.
- **Multimodal Approaches:** Integrating multiple modalities (e.g., visual, audio, textual) to improve detection accuracy by capturing inconsistencies across different types of data associated with a deep fake.
- **Adversarial Defence:** Developing robust detection methods that can withstand evasion techniques used by creators of deep fakes, such as adversarial training and testing against sophisticated attacks.

Real-time Detection: Researching techniques to perform detection in real-time or near real-time.

1.4 Existing System

Deep fake detection has evolved significantly, but older features that were commonly used include:

Inconsistencies in Facial Features: Detecting mismatches in facial expressions, eye movements, or lip sync between frames.

Statistical Analysis of Face and Body Movements: Analysing statistical patterns in facial movements or body gestures that might be unnatural.

Anomalies in Audio-Visual Synchronization: Checking for discrepancies between audio and video tracks, which may indicate manipulation.

Inconsistencies in Background or Scene: Identifying discrepancies in lighting, shadows, or reflections that suggest composite scenes.

Artificial fact Detection: Looking for compression artificial facts or other digital traces left by editing tools.

Biometric Markers: Using biometric features like blink rate, heart rate variability, or skin texture analysis to detect anomalies.

Metadata Analysis: Examining metadata embedded in digital files for inconsistencies or signs of tampering

1.5 Proposed System

Detecting deep fakes is an evolving field with several advanced features and techniques being developed to combat increasingly sophisticated methods used in creating them. Here are some advanced features used in deep fake detection:

Feature-based Analysis: Using advanced machine learning models to analyse specific features such as facial expressions, eye movements, and lip-syncing to detect inconsistencies or anomalies that suggest manipulation.

Temporal Analysis: Examining the temporal consistency of videos to detect abrupt changes or inconsistencies that are indicative of deep fakes. This involves analysing the flow of motion and ensuring it is consistent throughout the video.

GAN Identification: Generative Adversarial Networks (GANs) are often used to create deep fakes. Techniques involve identifying traces of GAN-generated artificial facts or patterns within images or videos.

Forensic Analysis: Applying digital forensic techniques to analyse metadata, compression artificial facts, and noise patterns within images or videos to detect traces left by editing tools or algorithms used in creating deep fakes.

Behavioural Analysis: Assessing behavioural cues such as blinking patterns, head movements, or natural interactions within videos to determine authenticity.

Capsule Networks: Utilizing capsule networks, an advanced type of neural network architecture, to better capture hierarchical relationships between features in images or videos, potentially improving detection accuracy.

Block chain and Watermarking: Implementing block chain technology or digital watermarking to track the origin and authenticity of media files, providing a way to verify if content has been tampered with.

CHAPTER 2

LITERATURE REVIEW

A literature survey on a deep fake detection project involves a comprehensive review of existing research, methods, and technologies related to detecting deep fakes. This survey provides a foundation for understanding the current state of the field, identifying gaps, and guiding the development of new detection techniques. Below is a structured approach to conducting a literature survey on deep fake detection:

1. Introduction

Purpose

Explain the significance of detecting deep fakes and the motivation behind the survey.

Scope

Outline the areas covered, such as image and video analysis, machine learning methods, datasets, and challenges.

2. Overview of Deep Fakes:

Definition:

Describe what deep fakes are and how they are created, typically using Generative Adversarial Networks (GANs).

Applications:

Mention both positive applications (e.g., in entertainment) and negative implications (e.g., misinformation)

3. Detection Techniques

Convolutional Neural Networks (CNNs):

Methodology

Explain how CNNs are used to analyse spatial features in images.

Examples

Meso Net focuses on mesoscopic properties for detection.

Conclusion:

Summary -Recap the key points covered in the survey.

Significance - Emphasize the importance of ongoing research in deep fake detection.

Call to Action - Encourage further research and collaboration across disciplines.

A literature review on deep fake detection encompasses a broad exploration of various methodologies, advancements, challenges, and future directions in the field. Here's a structured overview based on current research:

Introduction to Deep fakes and Their Impact:

Definition of deep fakes and their significance in media manipulation and misinformation.

Examples of notable deep fake incidents and their consequences.

Detection Techniques:

Digital Forensics: Analysis of metadata, compression artifacts, and inconsistencies in image or video files.

Biometric Analysis: Comparison of facial features, speech patterns, and other biometric signatures against known authentic data.

Artificial Intelligence Approaches: Application of machine learning models, including deep neural networks, for anomaly detection in media.

Behavioural Analysis: Examination of behavioural cues such as facial expressions, eye movements, and speech characteristics.

Advancements in Detection Models:

Generative Models: Understanding how generative adversarial networks (GANs) and other deep learning architectures are used to create deep fakes, and subsequently, how similar models can be employed for detection.

Feature Extraction: Techniques for extracting robust features from media files to detect manipulation.

Ensemble Methods: Combining multiple detection techniques or models to improve accuracy and reliability.

Challenges and Limitations:

Adversarial Techniques: Countermeasures employed by deep fake creators to evade detection.

Generalization and Transferability: Ensuring detection models generalize across different types of deep fakes and datasets.

Real-Time Detection: Developing systems capable of detecting deep fakes in real-time, considering the rapid evolution of deep fake technology.

CHAPTER 3

METHODOLOGIES

3.1 System Architecture

3.1.1 How deep-fakes are produced

Deep-fake models are typically created using deep learning techniques, particularly generative adversarial networks (GANs) and auto encoder-based architectures.

1) Data Collection: Deep-fake models require large datasets of images or videos of the target person(s) to be manipulated. These datasets serve as the basis for learning the facial features, expressions, and mannerisms that the model will replicate.

2) Preprocessing: The collect data is preprocessed to extract facial landmarks, align faces, and normalize lighting conditions. Preprocessing helps ensure consistency and quality in the training data.

3) Training the Generative Model: Generative adversarial networks (GANs) are commonly used for deep-fake generation. GANs consist of two neural networks: a generator and a discriminator. The generator learns to generate realistic fake images/videos from random noise, while the discriminator learns to distinguish between real and fake images/videos.

4) Training Process: The generator generates fake images/videos using random noise as input. The discriminator is trained to differentiate between real and fake images/videos. The generator and discriminator are trained iteratively in an adversarial manner. The generator tries to generate increasingly realistic images/videos to fool the discriminator, while the discriminator tries to become better at distinguishing between real and fake images/videos.

5) Loss Function: The training process is guided by a loss function that measures how well the generator is fooling. Common loss functions include binary cross entropy or Wasserstein distance.

6) Fine-Tuning and Refinement: After the initial training, the model may undergo fine-tuning and refinement to improve the quality of generated deep-fakes. This may involve adjusting hyper parameters, using larger datasets, or employing advanced techniques such as self-attention mechanisms.

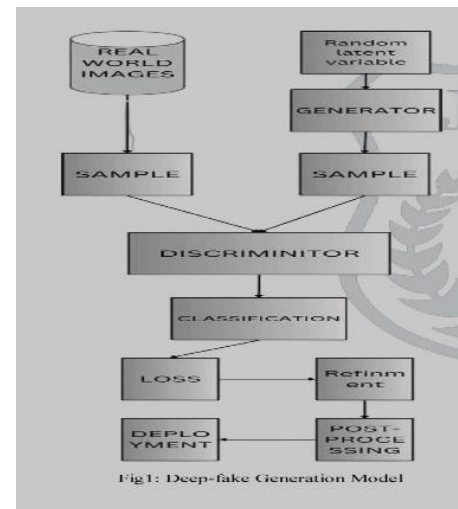
7) Post-Processing: The generated deep-fakes may undergo post-processing to enhance their realism. This could include adding imperfections, adjusting lighting, or smoothing transitions between frames.

8) Deployment: Once trained, the deep-fake model can be deployed to generate manipulated images or videos. These deep fakes can be shared online or used for various purposes, ranging from entertainment to malicious deception.

3.1.2 How can we detect deep-fakes?

According to MIT media lab there are several Deep-fake artifacts that we can be on the lookout for. When scrutinizing for deep-fakes, focus on facial features, particularly the eyes, cheeks, and forehead, assessing for inconsistencies such as overly smooth or wrinkled skin. Check for natural shadows and lighting reflections, especially in glasses, ensuring they align with movement. Examine facial hair, moles, and blinking patterns for authenticity. Pay close attention to lip movements, as discrepancies in syncing may indicate manipulation.

Fig 3.1.2: deep-fake generational model



3.1.3 Deep-fake detection mathematical model?

The mathematical model we used involves several components:

- a) **Face Detection with MTCNN:** Multi-task Cascaded Convolutional Networks (MTCNN) is used for face detection. MTCNN is a framework developed as a solution for both face detection and face alignment. The MTCNN model detects faces in an image by applying a series of convolutional filters to identify facial landmarks and bounding boxes.



Fig:3.1.3 (a): process of MTCNN

- b) **Face Recognition with InceptionResnetV1:** The InceptionResnetV1 model is used for face recognition. This model takes a cropped face image as input and outputs a feature vector that represents the characteristics of the face.

c) Grad CAM for Explainability: Grad CAM (Gradient-weighted Class Activation Mapping) is used to visualize the regions of the input image that are most influential in the model's decision-making process. It computes the gradients of the target class output with respect to the feature maps of the specified layers in the model. This helps in understanding which parts of the image contribute the most to the model's prediction.

d) Sigmoid Activation for Classification: The output of the model is passed through a sigmoid activation function, which squashes the output values between 0 and 1. This output represents the confidence score for the input image being classified as a real or fake face. Sigmoid activation function is used in classification:

- **Output Interpretation:** The output of the sigmoid function represents the probability or confidence score that an input belongs to the positive class (e.g., class 1). For binary classification, this is often interpreted as the probability of the input belonging to the "positive" class.

- **Decision Boundary:** In binary classification, a decision boundary is typically set at 0.5. Inputs with sigmoid output greater than 0.5 are classified as belonging to the positive class, while inputs with output less than 0.5 are classified as belonging to the negative class.

- **Training:** During the training process, the sigmoid function is applied to the output of the last layer of the neural network. The parameters of the network (weights and biases) are adjusted using optimization algorithms such as gradient descent to minimize the difference between the predicted outputs and the true labels of the training data.

- **Loss Function:** The output of the sigmoid function is often used as input to a loss function, such as binary cross-entropy loss, which quantifies the difference between the predicted probabilities and the true labels. This loss is minimized during training to improve the model's ability to correctly classify inputs.

e) Confidence Scores: The confidence scores for real and fake faces are computed based on the sigmoid output. The score closer to 1 indicates higher confidence in the predicted class. The confidence scores for classifying whether the input image contains a real or fake face are calculated using a sigmoid activation function applied to the output of the face recognition model. Here's how we have computed:

```

interface = gr.Interface(
    fn=predict,
    inputs=[
        gr.inputs.Image(label="Input Image", type="pil")
    ],
    outputs=[
        gr.outputs.Label(label="Class"),
        gr.outputs.Image(label="Face with Explainability", type="pil")
    ],
).launch()

```

Fig3.1.3 (c) : Code for confidence Scores

Output contains the sigmoid-activated output of the face recognition model. The sigmoid function squashes the raw output values to the range= [0, 1], representing the confidence scores for each class (real or fake).

- Real prediction:** represents the confidence score for the image being classified as "real" face.
- Fake prediction:** represents the confidence score for the image being classified as "fake" face.

f) Image Processing: Various image processing techniques such as resizing, normalization, and overlaying the Grad CAM heatmap on the original image are used for visualization and interpretability. Image processing steps involved:

Face Detection Preprocessing:

- The input image is passed through the MTCNN (Multi-task Cascaded Convolutional Networks) model for face detection.
- The detected face region is cropped and resized to a fixed size (256x256) using bilinear interpolation.

Visualization:

- The pre-processed face image is converted to a NumPy array for visualization.
- Before passing the face image to the model for prediction, it's normalized by dividing by 255.0 to scale pixel values between 0 and 1.

Grad CAM Visualization:

- Grad CAM is applied to the pre-processed face image to generate a heatmap highlighting the important regions for the model's prediction.
- The heatmap is overlaid onto the original face image to visualize which parts of the face are contributing the most to the model's decision.

3.2 Algorithms

Detecting deep fakes involves using various algorithms and techniques that leverage both visual and audio cues to identify inconsistencies or artifacts indicative of manipulation. Here's an overview of key algorithms commonly used in deep fake detection:

- **Face Detection and Recognition:**

Face Detection Algorithms: Algorithms like Haar cascades, Histogram of Oriented Gradients (HOG), or deep learning-based methods (e.g., Single Shot Multi box

Face Recognition: Techniques such as Eigen faces, Local Binary Patterns (LBP), or deep neural networks (e.g., VGG-Face, Face Net) are applied to verify the identity of detected faces. Detector - SSD, Faster R-CNN) are used to detect faces in images or videos.

- **Facial Manipulation Detection:**

Feature-Based Methods: Analyze facial landmarks, texture patterns, and symmetry to detect anomalies or inconsistencies that suggest facial manipulation

CNN-Based Approaches: Use convolutional neural networks (CNNs) to learn and detect artifacts specific to face swapping or facial reenactment techniques.

- **Temporal Analysis:**

Motion Analysis: Analyze the temporal coherence of facial movements, head pose changes, and eye blinking patterns to detect unnatural transitions or glitches.

Long Short-Term Memory (LSTM): Recurrent neural networks like LSTMs can model temporal dependencies in video sequences, aiding in detecting spatiotemporal inconsistencies.

- **Audio-Visual Synchronization:**

Lip Sync Detection: Compare lip movements extracted from video frames with corresponding audio features (e.g., spectrograms, Mel-frequency cepstral coefficients - MFCCs) to detect discrepancies that may indicate tampering.

- **Quality Assessment and Artifact Detection:**

Image Quality Metrics: Compute metrics such as noise levels, compression artifacts, or sharpness variations to identify anomalies introduced during the manipulation process.

Forgery Detection: Utilize algorithms to detect specific tampering artifacts like boundary artifacts, color inconsistencies, or inconsistent lighting.

- **Machine Learning and Deep Learning Models:**

Supervised Learning: Train classifiers (e.g., SVM, Random Forests, deep neural networks) using labeled datasets to distinguish between real and fake media.

Generative Models: Utilize generative adversarial networks (GANs) to generate synthetic samples for training or simulate potential deep fake creation methods for detection purposes.

- **Ensemble Methods and Fusion Techniques:**

Combination of Algorithms: Combine outputs from multiple detectors (e.g., visual, audio) using ensemble methods (e.g., voting, stacking) to enhance detection accuracy.

Decision Fusion: Integrate results from different modalities (e.g., visual features, audio features) to make a final decision on the authenticity of media content.

- **Post-Processing and Validation:**

Filtering and Refinement: Apply post-processing techniques to refine detection results and reduce false positives.

Validation Techniques: Use validation sets or cross-validation methods to assess the robustness and generalization ability of the detection algorithm.

CHAPTER 4

SOFTWARE REQUIREMENTS SPECIFICATIONS

4.1 SOFTWARE REQUIREMENTS

1. Operating System:

Linux (Ubuntu, CentOS): Preferred for its robustness and compatibility with various machine learning libraries.

Windows: An alternative for development, especially if using certain proprietary software.

2. Programming Languages:

Python: Primary language due to its extensive libraries for machine learning and data processing.

3. Libraries:

Machine Learning Frameworks

TensorFlow or PyTorch: For building and training deep learning models.

Keras : High-level API for TensorFlow.

Data Processing Libraries:

OpenCV: For image

4. Development Tools:

Integrated Development Environment (IDE): Pycharm, VS Code, or Jupyter Notebook: For development and testing.

Version Control:

Git: For version control and collaboration.

GitHub or GitLab: For repository hosting.

Detecting deep fakes requires a combination of software tools and techniques that typically involve:

Image/Video Analysis Libraries: Utilizing libraries like OpenCV, TensorFlow, PyTorch, or Dlib for image and video processing, which are capable of extracting frames, analyzing pixel data, and detecting anomalies.

Machine Learning Models: Implementing pre-trained or custom machine learning models (often convolutional neural networks - CNNs) trained specifically for deep fake detection. Models like Xception, VGG, or custom architectures trained on datasets containing deep fakes are commonly used.

Feature Extraction: Techniques to extract unique features from images or videos that may indicate manipulation or anomalies, such as inconsistencies in facial expressions, unnatural movements or discrepancies in lighting or shadowing pattern.

Dlib: Known for its facial recognition capabilities, which can be adapted for detecting facial inconsistencies in deep fake images.

Machine Learning Models:

Convolutional Neural Networks (CNNs): Specifically designed architectures that excel in image analysis tasks, capable of learning complex patterns and anomalies indicative of deep fake manipulation.

Facial Landmarks and Expressions: Algorithms that analyze facial landmarks, changes in expressions, blinking patterns, etc., to detect unnatural facial movements or inconsistencies.

4.2- HARDWARE REQUIREMENTS

Deep fakes effectively often requires hardware that Detecting can handle intensive computational tasks, especially when processing high-resolution videos or large datasets. Here are the key hardware requirements for deep fake detection systems:

- **Central Processing Unit (CPU):**

High-performance CPUs: Modern CPUs with multiple cores and high clock speeds are beneficial for tasks such as image/video processing, feature extraction, and running machine learning algorithms.

Multi-core Processing: Deep fake detection algorithms can leverage parallel processing capabilities of multi-core CPUs to speed up computations.

- **Graphics Processing Unit (GPU):**

CUDA-capable GPUs: NVIDIA GPUs with CUDA support are commonly used for deep learning tasks due to their ability to parallel computations.

Fast Storage: SSDs (Solid State Drives) are preferred over traditional HDDs (Hard Disk Drives) due to their faster read/write speeds, which are crucial for handling large volumes of media data efficiently.

Sufficient Storage Capacity: Deep fake detection systems may require substantial storage capacity to store datasets, models, and intermediate results, depending on the scale of analysis.

Network Interface:

High-speed Network Connectivity: For systems utilizing cloud-based deep learning services or distributed computing, a fast and reliable internet connection is essential for data transfer and model training/inference.

Specialized Hardware (Optional):

FPGA (Field-Programmable Gate Array): Can be used for accelerating specific tasks in deep fake detection through hardware acceleration.

ASICs (Application-Specific Integrated Circuits): Custom-designed chips optimized for deep learning tasks, providing high efficiency and performance in inference tasks.

Efficient Cooling Systems: Ensuring that the hardware setup is adequately cooled to maintain optimal performance during intensive computational tasks.

Stable Power Supply: Reliable power supply to prevent interruptions during prolonged computations, especially important for GPU-intensive tasks.

Scalability Considerations:

Cluster or Cloud Infrastructure: For large-scale deep fake detection applications, scalable solutions involving clusters of CPUs/GPUs or cloud-based services (e.g., AWS, Google Cloud) can provide flexibility and cost-effectiveness.

Choosing the right hardware configuration depends on factors such as the scale of detection tasks, real-time requirements, and budget constraints. Many deep fake detection systems leverage a combination of CPU and GPU resources to balance computational power and cost-effectiveness, ensuring efficient and accurate detection of manipulated media contents

Minimum configuration

CPU: Intel i3 5th Gen

RAM: 8 GB

Storage: 256 GB SSD (for faster data access and storage)

Networking: High-speed internet connection (preferably wired for stability)

Networking :

High-speed Internet Connection :

- Essential for real-time data ingestion and interaction with cloud services.

Ethernet Connectivity :

- Prefer wired connections over wireless for lower latency and higher stability, especially for server setups.

4.3 LANGUAGES / TECHNOLOGIES USED

Deep fake detection, particularly for images, involves a range of languages and technologies. Here's an overview of the common ones used in this field:

Programming Languages:

Python: The primary language due to its extensive libraries and frameworks for machine learning, deep learning, and image processing.

Python is a widely used general-purpose, high-level programming language with design philosophy emphasizing code readability. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Its syntax allows programmers to express concepts in fewer lines of code. Some features of Python:

- Easy To Learn & Use While Coding.
- Extensible Feature.
- Interpreted Language.
- Expressive Language.
- Cross-Platform Portable Language.
- Dynamic Memory Allocation.
- High-Level Interpreted Language.
- Graphical User Interface (GUI) Support

Python has specific functionality and is convenient to be used for both quantitative and analytical computational purposes. Data Science Python is popularly used and, as well as being a dynamic and open-

source language, is a top choice. Its massive libraries are also used to manipulate the data however for a beginner data analyst they are really simple to learn. The python libraries used in this thesis are briefly described as follows:

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.

Generative Adversarial Network:

Generative Adversarial Networks (GANs) are a powerful class of neural networks that are used for an [unsupervised learning](#). GANs are made up of two [neural networks](#), a **discriminator** and a **generator**. They use adversarial training to produce artificial data that is identical to actual data.

- The Generator attempts to fool the Discriminator, which is tasked with accurately distinguishing between produced and genuine data, by producing random noise samples.
- Realistic, high-quality samples are produced as a result of this competitive interaction, which drives both networks toward advancement.

Jupyter Notebook:

The Jupyter Notebook is an incredibly powerful tool for interactively developing and presenting data science projects. A notebook integrates code and its output into a single document that combines visualisations, narrative text, mathematical equations, and other rich media. The intuitive workflow promotes iterative and rapid development, making notebooks an increasingly popular choice at the heart of contemporary data science, analysis, and increasingly science at large. Best of all, as part of the open source [Project Jupyter](#), they are completely free.

CHAPTER – 5

IMPLEMENTATION AND RESULT

5.1 Code

```
import gradio as gr
import torch
import torch.nn.functional as F
from facenet_pytorch import MTCNN, InceptionResnetV1
import numpy as np
from PIL import Image
import cv2
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
from pytorch_grad_cam.utils.image import show_cam_on_image
import warnings
warnings.filterwarnings("ignore")

Download and Load Model

DEVICE = 'cuda:0' if torch.cuda.is_available() else 'cpu'

mtcnn = MTCNN(
    select_largest=False,
    post_process=False,
    device=DEVICE
).to(DEVICE).eval()

model = InceptionResnetV1(
    pretrained="vggface2",
    classify=True,
    num_classes=1,
    device=DEVICE
)

checkpoint = torch.load("resnetinceptionv1_epoch_32.pth", map_location=torch.device('cpu'))
```

```

model.load_state_dict(checkpoint['model_state_dict'])

model.to(DEVICE)

model.eval()

InceptionResnetV1(
  (conv2d_1a): BasicConv2d(
    (conv): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (conv2d_2a): BasicConv2d(
    (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (conv2d_2b): BasicConv2d(
    (conv): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (maxpool_3a): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2d_3b): BasicConv2d(
    (conv): Conv2d(64, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(80, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (conv2d_4a): BasicConv2d(
    (conv): Conv2d(80, 192, kernel_size=(3, 3), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

```

```

(relu): ReLU()

)

(conv2d_4b): BasicConv2d(

  (conv): Conv2d(192, 256, kernel_size=(3, 3), stride=(2, 2), bias=False)

  (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

  (relu): ReLU()

)

(repeat_1): Sequential(

  (0): Block35(

    (branch0): BasicConv2d(

      (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

      (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

      (relu): ReLU()

    )

    (branch1): Sequential(

      (0): BasicConv2d(

        (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

        (relu): ReLU()

      )

      (1): BasicConv2d(

        (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

        (relu): ReLU()

      )

    )

  )

  (branch2): Sequential(

    (0): BasicConv2d(

```

```

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(2): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

)

(conv2d): Conv2d(96, 256, kernel_size=(1, 1), stride=(1, 1))

(rel): ReLU()

)

(1): Block35(

(branch0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(branch1): Sequential(

(0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

```

```

    (relu): ReLU()
)
(1): BasicConv2d(
  (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
(branch2): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  )
  )
  (conv2d): Conv2d(96, 256, kernel_size=(1, 1), stride=(1, 1))
  (relu): ReLU()
)

```

(2): Block35(

(branch0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(branch1): Sequential(

(0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

)

(branch2): Sequential(

(0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

```

)

(2): BasicConv2d(
  (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)

(conv2d): Conv2d(96, 256, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()
)

(3): Block35(
  (branch0): BasicConv2d(
    (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (1): BasicConv2d(
      (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
  )
)

```

```

(branch2): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  )
  (conv2d): Conv2d(96, 256, kernel_size=(1, 1), stride=(1, 1))
  (relu): ReLU()
)
(4): Block35(
  (branch0): BasicConv2d(
    (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(

```

```

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

)

(branch2): Sequential(

(0): BasicConv2d(

(conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(2): BasicConv2d(

(conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

(bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

)

(conv2d): Conv2d(96, 256, kernel_size=(1, 1), stride=(1, 1))

```

```

    (relu): ReLU()
)
)
(mixed_6a): Mixed_6a(
  (branch0): BasicConv2d(
    (conv): Conv2d(256, 384, kernel_size=(3, 3), stride=(2, 2), bias=False)
    (bn): BatchNorm2d(384, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(256, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (1): BasicConv2d(
      (conv): Conv2d(192, 192, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (2): BasicConv2d(
      (conv): Conv2d(192, 256, kernel_size=(3, 3), stride=(2, 2), bias=False)
      (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
  )
  (branch2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
)

```

```

(repeat_2): Sequential(
  (0): Block17(
    (branch0): BasicConv2d(
      (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (branch1): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU()
      )
      (1): BasicConv2d(
        (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU()
      )
      (2): BasicConv2d(
        (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU()
      )
    )
    (conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
    (relu): ReLU()
  )
)
(1): Block17(

```

```

(branch0): BasicConv2d(
  (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)
(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(2): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)

(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()

)

(3): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()

```

```

)
(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)
)
(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(4): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(

```

```

(0): BasicConv2d(
  (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(1): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(2): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
)
(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(5): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

(1): BasicConv2d(

    (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)

    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

(2): BasicConv2d(

    (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)

    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

)

(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()

)

(6): Block17(

    (branch0): BasicConv2d(

        (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

        (relu): ReLU()

    )

    (branch1): Sequential(

        (0): BasicConv2d(

            (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

            (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

            (relu): ReLU()

```

```

)

(1): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)

(2): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)

)

)

(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()

)

(7): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (1): BasicConv2d(

```

```

(conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(2): BasicConv2d(

(conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

)

(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))

(rel): ReLU()

)

(8): Block17(

(branch0): BasicConv2d(

(conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(branch1): Sequential(

(0): BasicConv2d(

(conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(rel): ReLU()

)

(1): BasicConv2d(

(conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)

(bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

```

```

    (relu): ReLU()
)
(2): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(9): Block17(
  (branch0): BasicConv2d(
    (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(896, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (1): BasicConv2d(
      (conv): Conv2d(128, 128, kernel_size=(1, 7), stride=(1, 1), padding=(0, 3), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
  )
)

```

```

(2): BasicConv2d(
  (conv): Conv2d(128, 128, kernel_size=(7, 1), stride=(1, 1), padding=(3, 0), bias=False)
  (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
(conv2d): Conv2d(256, 896, kernel_size=(1, 1), stride=(1, 1))
  (relu): ReLU()
)
)
(mixed_7a): Mixed_7a(
  (branch0): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(896, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
    (1): BasicConv2d(
      (conv): Conv2d(256, 384, kernel_size=(3, 3), stride=(2, 2), bias=False)
      (bn): BatchNorm2d(384, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()
    )
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(896, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU()

```

```

)
(1): BasicConv2d(
  (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), bias=False)
  (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
(branch2): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(896, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), bias=False)
    (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)
(branch3): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(repeat_3): Sequential(
  (0): Block8(

```

```

(branch0): BasicConv2d(
  (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)
(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(1): Block8(
  (branch0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

(bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(relu): ReLU()

)

(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)

(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()

)

(2): Block8(
  (branch0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()

```

```

)
(branch1): Sequential(
  (0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (1): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (2): BasicConv2d(
    (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
)
)
(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(3): Block8(
  (branch0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(

```

```

(0): BasicConv2d(
  (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(1): BasicConv2d(
  (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)
  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
(2): BasicConv2d(
  (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)
  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU()
)
)
)
(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))
(relu): ReLU()
)
(4): Block8(
  (branch0): BasicConv2d(
    (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU()
  )
  (branch1): Sequential(
    (0): BasicConv2d(
      (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

(1): BasicConv2d(

    (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)

    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

(2): BasicConv2d(

    (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)

    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

    (relu): ReLU()

)

)

(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))

(relu): ReLU()

)

)

(block8): Block8(

    (branch0): BasicConv2d(

        (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)

        (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

        (relu): ReLU()

    )

    (branch1): Sequential(

        (0): BasicConv2d(

            (conv): Conv2d(1792, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)

            (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

```

```

(relu): ReLU()

)

(1): BasicConv2d(

  (conv): Conv2d(192, 192, kernel_size=(1, 3), stride=(1, 1), padding=(0, 1), bias=False)

  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

  (relu): ReLU()

)

(2): BasicConv2d(

  (conv): Conv2d(192, 192, kernel_size=(3, 1), stride=(1, 1), padding=(1, 0), bias=False)

  (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

  (relu): ReLU()

)

)

(conv2d): Conv2d(384, 1792, kernel_size=(1, 1), stride=(1, 1))

)

(avgpool_1a): AdaptiveAvgPool2d(output_size=1)

(dropout): Dropout(p=0.6, inplace=False)

(last_linear): Linear(in_features=1792, out_features=512, bias=False)

(last_bn): BatchNorm1d(512, eps=0.001, momentum=0.1, affine=True, track_running_stats=True)

(logits): Linear(in_features=512, out_features=1, bias=True)

def predict(input_image:Image.Image):

    """Predict the label of the input_image"""

    face = mtcnn(input_image)

    if face is None:

        raise Exception("No face detected")

    face = face.unsqueeze(0) # add the batch dimension

    face = F.interpolate(face, size=(256, 256), mode='bilinear', align_corners=False)

```

```

# convert the face into a numpy array to be able to plot it

prev_face = face.squeeze(0).permute(1, 2, 0).cpu().detach().int().numpy()

prev_face = prev_face.astype('uint8')


face = face.to(DEVICE)

face = face.to(torch.float32)

face = face / 255.0

face_image_to_plot = face.squeeze(0).permute(1, 2, 0).cpu().detach().int().numpy()


target_layers=[model.block8.branch1[-1]]

use_cuda = True if torch.cuda.is_available() else False

cam = GradCAM(model=model, target_layers=target_layers, use_cuda=use_cuda)

targets = [ClassifierOutputTarget(0)]


grayscale_cam = cam(input_tensor=face, targets=targets, eigen_smooth=True)

grayscale_cam = grayscale_cam[0, :]

visualization = show_cam_on_image(face_image_to_plot, grayscale_cam, use_rgb=True)

face_with_mask = cv2.addWeighted(prev_face, 1, visualization, 0.5, 0)


with torch.no_grad():

    output = torch.sigmoid(model(face).squeeze(0))

    prediction = "real" if output.item() < 0.5 else "fake"


real_prediction = 1 - output.item()

fake_prediction = output.item()


confidences = {'real': real_prediction,

               'fake': fake_prediction

```

```

    }

return confidences, face_with_mask

Gradio Interface

interface = gr.Interface(

fn=predict,

inputs=[

    gr.inputs.Image(label="Input Image", type="pil")

],

outputs=[

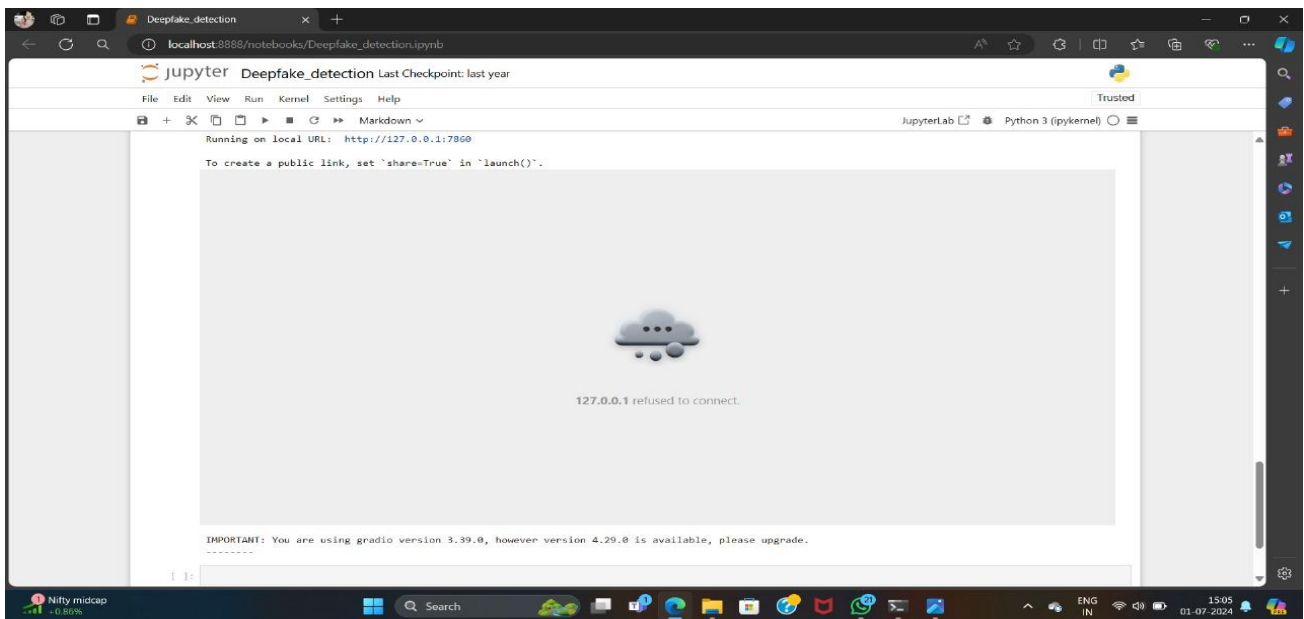
    gr.outputs.Label(label="Class"),

    gr.outputs.Image(label="Face with Explainability", type="pil")

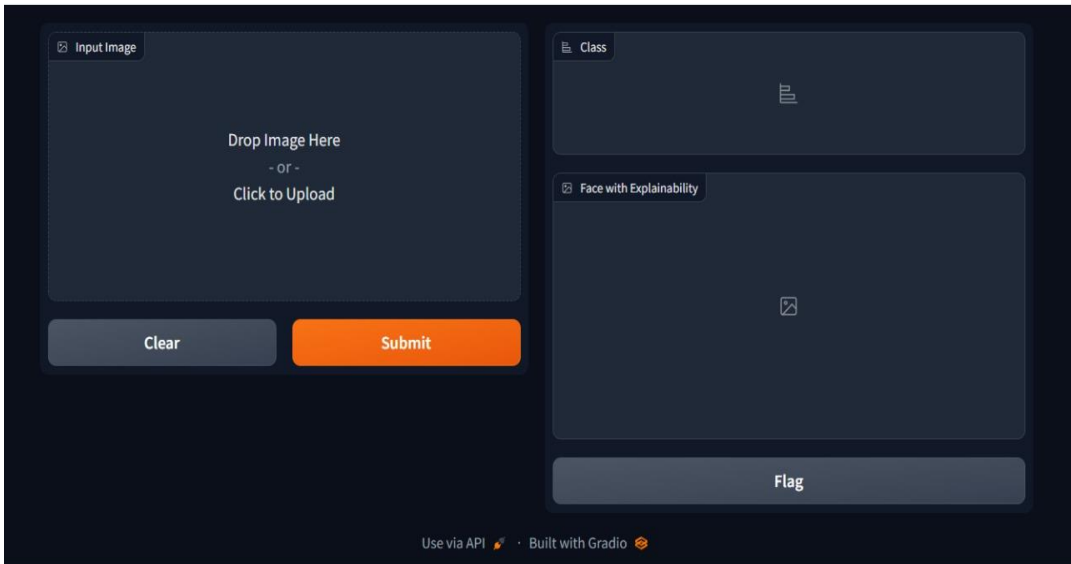
],

).launch()

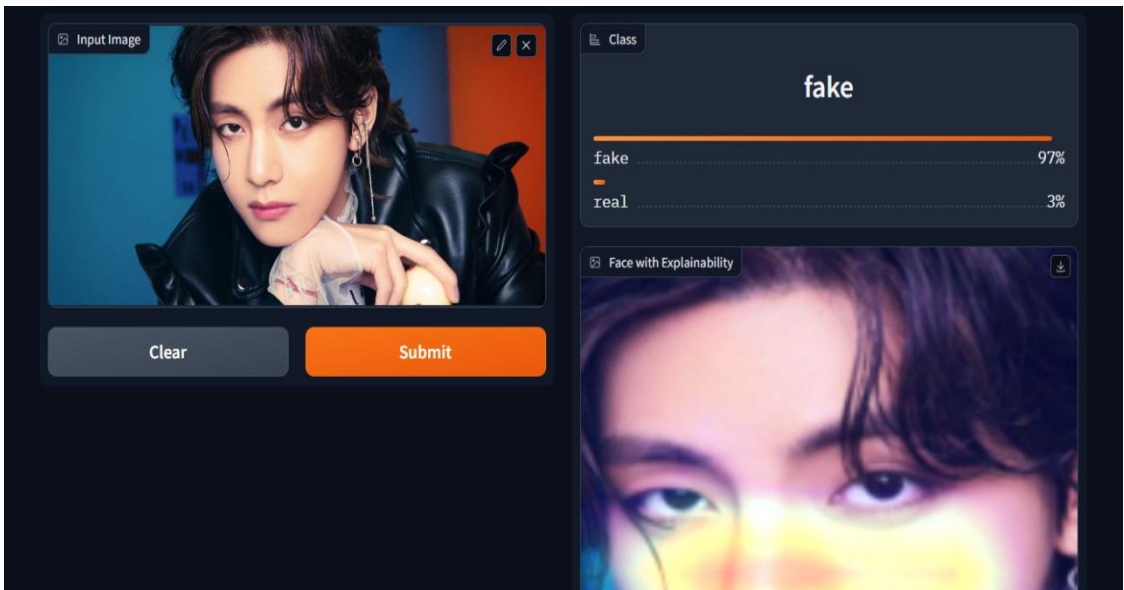
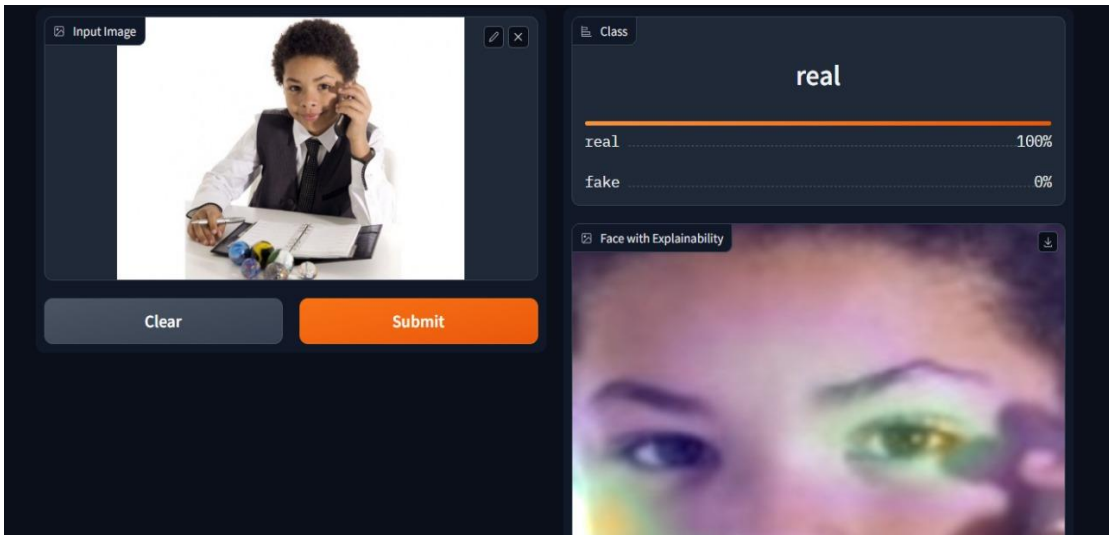
```



5.2 Output



IMPORTANT: You are using gradio version 3.39.0, however version 4.29.0 is available, please upgrade.



CHAPTER – 06

RESULT

6.1 Real image Input:

The input to the deep-fake detection model is an image containing a face. This image is expected to be in a format compatible with the gradio library's Image input type, which typically accepts PIL (Python Imaging Library) Image objects. Here are the details of the input:

- Type: Image (PIL format)
- Content: The input image should contain a single face that needs to be classified as real or fake.



Fig6.1: Example input image.

6.2 Output for real images:

The output of the deep-fake detection model consists of two components: a label indicating whether the face is predicted as real or fake, and an image showing the input face with an overlaid heatmap for explanation. Here are the details of the output:

Label:

- Type: String
- Content: The predicted label indicating whether the input face is classified as "real" or "fake".

Face with Explainability:

- Type: Image (PIL format)

- Content: An image showing the input face with an overlaid heatmap generated by Grad CAM. This heatmap highlights the regions of the face that are most influential in the model's decision-making process, providing an explanation for the prediction.

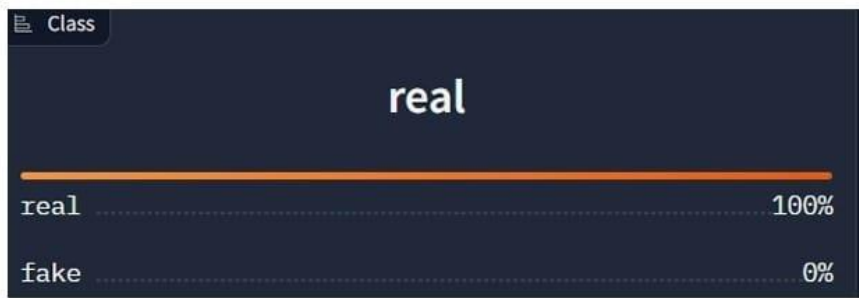


Fig6.2 (a) : Predicted label indication for fig6.1.

It says that the input image is 100% real and 0% fake. Therefore, we can confidently say that the input image is a legit image not a deep-fake image.

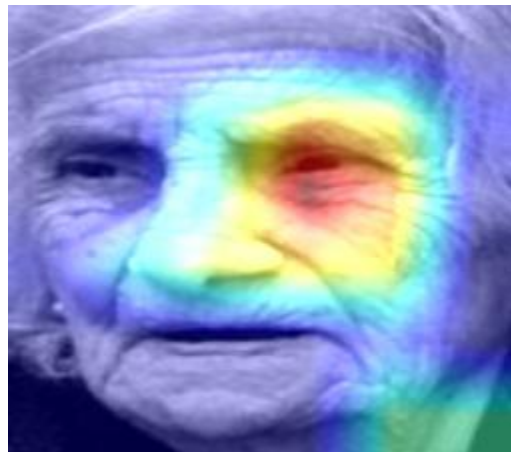


Fig6.2 (b): Input face with an overlaid heatmap generated by Grad CAM.

These were the predictions made by our deep-fake detection model. The photo is taken from one of his social media accounts.

6.3) Deep-fake Image Input:



Fig6.3: An AI generated image

6.4) Output images for fig6.3:

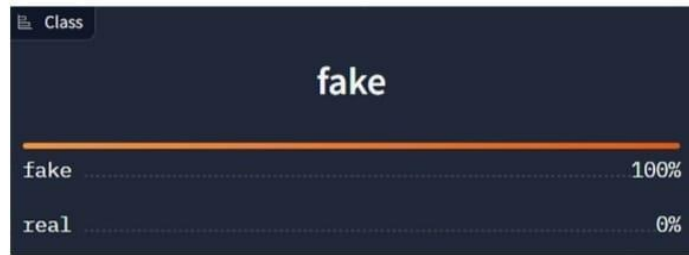


Fig6.4: Predicted label indication.

It says that the input image is 100% fake and 0% real. Therefore, we can confidently say that the input image is a not legit image not a deep-fake image.



Fig6.4 (a) : Input face with an overlaid heatmap generated by Grad CAM

CHAPTER – 07

CONCLUSION

In conclusion, this documentation provides a comprehensive overview of the deep-fake phenomenon, its implications, and the efforts to develop effective detection techniques using machine learning approaches. Deep-fake technology, driven by advances in deep learning algorithms, has raised significant concerns due to its potential for spreading misinformation and undermining trust in digital media. The documentation outlines the process of deep-fake generation, highlighting the use of generative adversarial networks (GANs) and deep neural networks to create hyper realistic manipulated media. It also discusses key indicators and artifacts that can be used to detect deep-fakes, emphasizing the importance of paying attention to facial features, expressions, and inconsistencies in the generated content. Furthermore, the documentation delves into the mathematical models and techniques employed for deep-fake detection, including face detection with MTCNN, face recognition with InceptionResnetV1, Grad CAM for explainability, and sigmoid activation for classification. These models, coupled with image processing techniques, enable the visualization and interpretation of deep-fake detection results. The applications of deep-fake detection span various domains, including media integrity, cybersecurity, forensic analysis, political disinformation, brand protection, and content moderation. By leveraging deep-fake detection technologies, stakeholders can mitigate the risks associated with the malicious use of synthetic media and safeguard the authenticity of digital content. Overall, this documentation underscores the importance of ongoing research and innovation in deep-fake detection to address the evolving threat landscape and preserve trust in digital communication channels. By advancing detection capabilities and addressing ethical considerations, we can better protect individuals, organizations, and society as a whole from the harmful effects of deep-fake manipulation.

CHAPTER – 08

FUTURE ENHANCEMENT

Advanced Model Architectures: Implement transformer-based models for improved context understanding and feature extraction.

Multi-Modal Detection: Integrate audio and visual data to enhance detection accuracy by analysing inconsistencies across modalities.

Improved Dataset Diversity: Expand datasets to include more varied scenarios, different cultures, and languages to improve generalization.

Synthetic Data Generation: Use GANs to create synthetic training data, addressing data scarcity and enhancing model robustness.

Explainable AI: Develop better explainability tools to understand and interpret model decisions, increasing trust and transparency.

Education and Awareness: Increasing public awareness about deep fakes and educating users about how to identify manipulated content through digital literacy programs and tools.

Real-Time Detection: Developing real-time detection systems that can quickly analyze and flag suspicious content as it is uploaded or shared online.

Privacy-Preserving Techniques: Developing privacy-preserving techniques that allow for deep fake detection without compromising the privacy of individuals whose data is used in training or testing the models.

Zero-Shot Learning: Advancing zero-shot learning techniques to detect deep fakes without the need for extensive training on specific types of deep fakes, thereby improving generalizability.

Regulatory Frameworks: Establishing regulatory frameworks and standards for deep fake detection technologies to ensure consistency, reliability, and ethical use across different jurisdictions.

Crowdsourcing and Citizen Science: Leveraging crowdsourcing and citizen science approaches to gather large-scale datasets and annotations for training and evaluating deep fake detection models.

Adversarial Robustness: Designing detection systems that are resilient to adversarial attacks, where deep fakes are intentionally crafted to evade detection algorithms.

CHAPTER – 09

REFERENCES

Assessment and Detection of Deepfakes: Korshunov, P., & Marcel, S. (2018). "Deepfakes: a New Threat to Face Recognition? Assessment and Detection." arXiv preprint arXiv:1812.08685. Retrieved from <https://arxiv.org/abs/1812.08685>.

Survey of Face Manipulation and Fake Detection Techniques: Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). "Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection." *Information Fusion*, 64, 131-148. Retrieved from <https://www.sciencedirect.com/science/article/pii/S156625352030243X>